基于梯度下降的线性回归拟合

一、方法介绍

1. 模型形式

数据里 x 是学习时长 hours，y 是成绩 score，只有一个特征，所以模型就是一条直线：y = w * x + b。代码里为了以后方便加特征，写成矩阵形式 y = x @ w + b。w 和 b 一开始都设为 0。

2. 损失函数

用均方误差 MSE：把每个样本的预测值和真实值之差平方，再求平均，写成公式就是 L = (1/n) * 求和 (y_hat - y)^2。平方有两个作用：一是正负误差不会互相抵消，二是错得越多的样本会被罚得越重。

3. 梯度下降和更新公式

损失对 w 和 b 求导（用链式法则推出来的）：

dL/dw = (2/n) * 求和 (y_hat - y) * x

dL/db = (2/n) * 求和 (y_hat - y)

然后让参数往梯度的反方向走一小步，反复迭代：

w = w - lr * dw

b = b - lr * db

lr 是学习率，决定每一步走多大。太小要走很多步，太大就会直接飞出去。

4. 批量大小的三种选择

代码里用“每一批取多少个样本”这一个参数，就实现了三种梯度下降：每批用全部 100 个样本叫批量梯度下降，算出来的梯度最准，但一个 epoch 只能更新 1 次；每批只用 1 个样本叫随机梯度下降，一个 epoch 能更新 100 次，但梯度很抖；每批 16 个是折中，也是实际最常用的做法。

5. 数据标准化

hours 大概在 1 到 10 之间，score 大概在 30 到 100 之间，两个数差了一个数量级。不做处理的话，损失函数的等高线会被拉成又细又长的峡谷，梯度下降在里面来回震荡，很难收敛。所以先把 x 变成“平均值 0、标准差 1”的形式：x = (x - mean) / std。注意测试集要用训练集的 mean 和 std，不然就等于提前偷看了测试集。

6. 数据划分

训练集 train.csv（100 条）只用来训练模型，测试集 test.csv（30 条）只在最后评估一次，不参与任何一次参数更新。

二、实验结果

1. 和闭式解对拍

一个特征的线性回归有一个精确解（闭式解），可以直接用公式算出来，不需要迭代。我用它来检验梯度下降对不对。注意模型是在标准化后的 x 上训练的，所以要先把参数换算回原始的小时才有意义：

闭式解：y = 6.1154 * hours + 33.3054

梯度下降（lr=0.1，每批 100 个，200 轮）：y = 6.1154 * hours + 33.3054

两个结果只差在小数点后很多位，说明梯度推导和代码都是对的。

2. 损失下降曲线和不同学习率的对比

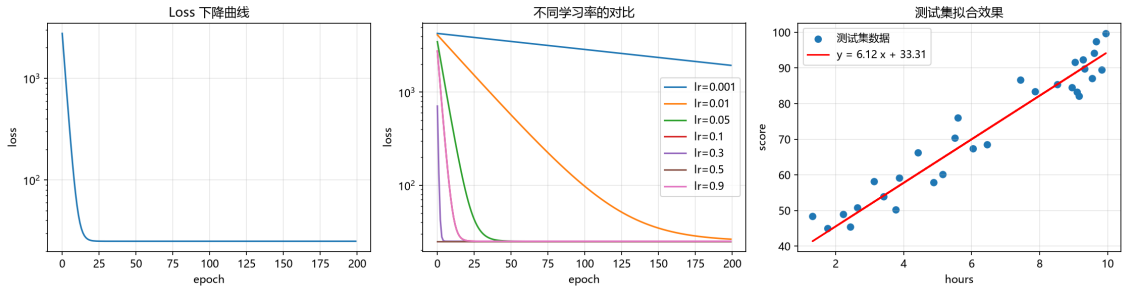


图 1 左：Loss 随 epoch 下降；中：不同学习率的对比；右：测试集上的拟合效果。

学习率	200 轮后的训练损失
lr = 0.001	1945.41
lr = 0.01	26.30
lr = 0.05	24.98
lr = 0.1	24.98
lr = 0.3	24.98
lr = 0.5	24.98
lr = 0.9	24.98
lr = 1.2	1.2e+62（已经发散）

从表里能看出三件事：学习率 0.001 太小了，200 轮之后损失还有 1945，离最低点差得远；学习率 0.05 到 0.9 都能收敛到 24.98（这就是理论最低值）；学习率 1.2 太大，损失直接涨到 1e62 然后变成 inf，模型完全废掉。

3. 不同批量大小的对比

每批样本数	每个 epoch 更新次数	200 轮后的损失
100	1	24.9786
16	6	25.0070

1	100	28.7995

每批 100 个的时候损失最低（24.9786），因为每次更新用的信息最全。每批 1 个的时候损失最差（28.7995），因为梯度只由一个样本决定，抖得厉害，所以到达最低点附近之后还会被单个样本带着来回跳。每批 16 个介于两者之间，这就是小批量梯度下降被广泛使用的原因。

4. 测试集上的拟合效果

最终模型：y = 6.1154 * hours + 33.3054

在训练集上的损失是 24.9786，在 30 条测试数据上：MSE = 20.1741，RMSE = 4.4916 分，R2 = 0.9328

R2 = 0.9328 表示模型解释了大约 93.3% 的成绩变化，RMSE = 4.4916 分表示平均预测误差大概 4.5 分。测试集的损失和训练集很接近，说明模型没有过拟合。图 1
右边可以看到，测试集的点基本都落在直线附近。

三、提问环节

问题 1：为什么一定要做标准化？

一开始我没做标准化，直接用原始的 hours 训练，学习率取 0.1 的时候损失很快就变成 nan。我以为是梯度写错了，但用数值微分检查过梯度之后发现没错。后来才想明白是数据量级的问题：hours 是 1 到 10，score 是 30 到 100，损失函数的等高线是一个很扁的椭圆，w 方向和 b 方向的曲率差了三十多倍，梯度下降在长轴方向容易震荡、在短轴方向又走得太慢。做了标准化之后，两个方向的尺度接近了，同一个学习率就能稳定收敛。

问题 2：学习率为什么不能太大？

实验里 lr = 0.9 还能收敛到最低点，lr = 1.2 就直接发散到
1e62。我的理解是：梯度下降每一步走 lr 乘梯度那么远，如果一步跨过了最低点、而且跨得比到最低点的距离还远，下一次就会跑到更远的地方，误差被越放越大，最后就飞出去了。

问题 3：小批量为什么损失更高？

每批只用 1 个样本的时候，最后损失是 28.80，比每批 100 个的 24.98 差不少。我一开始以为是小批量效果更差，后来想明白了：不是它学不到最优点，而是它每一步只看一个样本，方向很偏，走到最优点附近之后还会被单个样本带着来回跳，所以最后停下来的位置不一定是最好那个点。

问题 4：图被一个特别大的数毁掉了

第一次画“不同学习率对比”这张图的时候，图完全不能看：lr = 1.2 的损失是
1e62，这一个点把纵轴撑到了
1e62，其他几条正常曲线全被压成贴着横轴的一条直线。我试过的办法是把纵轴改成对数坐标，但 1e62 和 20 差得还是太多，最后只能在画图的时候把 lr = 1.2
这条线去掉。它的结果我保留在控制台输出和表里，这样信息没有丢，图也能看。

还没解决的问题

① 我是用训练集的损失来选学习率的，严格来说应该再切一份验证集专门调参，但数据只有 100 条，再切就更少了，所以这次没有做。② 画图那个问题其实没有真正解决，我只是把发散的那条线“藏起来”了。我还不知道怎样在一张图里既能看清正常曲线、又能看清爆炸的那条。